pgops — Plan

Deep dive · Technical sections · Non-technical sections

May 2026

pgops — modern Postgres ops toolkit

Plain English. A single TUI binary with subcommands for Postgres on-call workflows that don't have good OSS terminal tools — lock blockers, slow queries, autovacuum, replication slots, partitions, pre-flight migrations, EXPLAIN plans via local web, cloud-event correlation. Universal snapshot-to-markdown key and an MCP server surface for agents.

Shape. TS CLI built with Ink. Eight subcommands + Prometheus exporters + MCP server.

User and workflow

User. Postgres DBAs, SREs, backend engineers who get paged for the DB. Already live in the terminal — psql + bastion + kubectl in three tabs.

Task. Five workflows — incident triage, slow-API investigation, routine ops, migration safety, postmortem reconstruction.

Tools today. psql + saved CTEs, pgHero (Ruby web dashboard, daily), pgBadger (Perl offline log report), pganalyze (closed SaaS, $149/server/mo), pgcenter (Go TUI, general, slowing).

Where this fits. Replaces "SSH in, open psql, paste my saved CTE, scan by eye, write the kill statement by hand" with a single binary that owns the on-call workflow end-to-end.

Competitors and what we do differently

Tool	What it does	What's missing
pgHero	Free Ruby web dashboard for daily team checks	Web, not terminal. Need to host. Not incident-shaped.
pgBadger	Perl log-parser → HTML report	Offline. Useless during an active incident.
pganalyze	Paid SaaS with deep insights	Closed. Sends data outside your VPC. $149/server/mo.
pgcenter	Go TUI, general Postgres ops	Slowing maintenance. Many screens, none incident-shaped.

Different. Terminal-native, action-shaped (kill, snapshot, advise), free, focused per-pain. The frame:

> *"pgHero is your daily dashboard. pgBadger is your postmortem tool. pganalyze is the enterprise SaaS.* ***pgops is what you run during the incident.***"

User flow (what the user does)

Incident (the killer use case).

1. Page fires. SSH to bastion.

2. `pgops locks` — root blocker visible at the top of the chain; kill with `k`; snapshot with `s`.
3. `pgops events --around now-15m` — correlate with an RDS failover or maintenance window.
4. Paste the snapshot into the Slack incident channel.

Slow API.

1. APM flags endpoint X as slow.
2. `pgops top --table=events` — find the guilty query.
3. Enter → EXPLAIN tree. Complex plan? `pgops explain --serve` opens it in your browser.
4. Spot the missing index. Write the migration.

Migration.

1. `pgops review migrations/0042.sql` — pre-flight safety check.
2. Apply via Drizzle / Prisma / Liquibase.
3. `pgops locks` open in another tab — watch live during the apply.

Routine ops.

1. Morning: `pgops vacuum`, `pgops slots`.
2. Weekly: `pgops top` snapshot vs. last week.
3. Monthly: `pgops parts` to drop old + create new partitions.

Architecture (how the system works)

Stack and how it connects:

- **TS binary (`pgops`)** with a subcommand router. Single entry point, multiple commands.
- **node-postgres (`pg`)** for all DB connections. Reads `DATABASE_URL` from env or `-d` flag.
- **Ink (React for terminals)** for every TUI view — shared key handlers (`j/k`, `/`, `Enter`, `s` for snapshot, `q` to quit), shared theming, shared status bar.
- **Per-subcommand modules** — each subcommand is a folder with its own queries, render logic, tests. Develops in isolation.
- **Shared snapshot exporter** — any view can serialize to markdown via the `[s]` key. Cross-cutting capability, not a subcommand.
- **`@modelcontextprotocol/sdk`** for the optional MCP server (`pgops mcp`) that wraps the same query layer for agents.
- **Prometheus exporter siblings** (only for `vacuum` and `slots` — where trends matter) — standalone HTTP servers reusing the same queries.
- **AWS SDK + `@kubernetes/client-node`** as optional deps for `pgops events` (cloud event correlation).

Flow (illustrated with `pgops locks`):

```
User runs `pgops locks`
  → CLI router dispatches to locks subcommand
  → node-postgres connects from DATABASE_URL
  → every 2s:
      SELECT lock chain (pg_locks JOIN pg_stat_activity)
      → build root-blocker tree
      → Ink re-renders
  → on `k`:
      confirm modal → pg_terminate_backend(pid) → audit log
  → on `s`:
      shared snapshot exporter → markdown to stdout / file
  → on `q`:
      close connection, exit
```

Why these choices. One binary so cross-promotion compounds. Shared connection / Ink / snapshot code means each new subcommand is ~60% reused infrastructure. MCP as a thin wrapper over the same query layer — no duplicate logic. Prometheus exporters only for the two subcommands where trends matter; the rest are snapshot-shaped.

The strategic risk. pgcenter could speed up; pganalyze could open-source some features. Mitigation: stay narrow on the on-call frame, ship fast (v0 in 4 weeks, v1 in 12), let "terminal-native + action-shaped + free" be the moat.

Future features (roadmap)

- **Team snapshot history.** A `pgops history` subcommand that persists snapshots so postmortems pull from a real archive.
- **`pgops doctor`.** A meta-subcommand that runs every check and produces one ranked report — for routine ops review.
- **More cloud event correlators.** GCP, Azure, Aiven, Crunchy Bridge.
- **Anomaly alerts (lightweight).** Optional `pgops watch` daemon for specific signals — without trying to be a full Datadog.

Part 1 — Technical

A. Per-subcommand deep dives

One page per subcommand. Each includes the actual Postgres query, edge cases, render approach, and what to test.

A1. `pgops locks` — incident lock chain

Core query. Walks the wait graph: every blocked PID joined to whoever is blocking it.

```
WITH lock_chain AS (
  SELECT
    blocked.pid                 AS blocked_pid,
    blocking.pid                AS blocking_pid,
    blocked_act.usename         AS blocked_user,
    blocking_act.usename        AS blocking_user,
    blocked_act.application_name AS blocked_app,
    blocking_act.application_name AS blocking_app,
    blocked.locktype,
    blocked.mode                AS waiting_mode,
    blocking.mode               AS held_mode,
    blocked.relation::regclass  AS table_name,
    blocking_act.query          AS blocking_query,
    blocked_act.query           AS blocked_query,
    EXTRACT(EPOCH FROM (now() - blocked_act.xact_start))   AS blocked_for_secs,
    EXTRACT(EPOCH FROM (now() - blocking_act.xact_start))  AS blocking_for_secs
  FROM pg_locks blocked
  JOIN pg_stat_activity blocked_act  ON blocked.pid = blocked_act.pid
  JOIN pg_locks blocking
    ON blocked.locktype          IS NOT DISTINCT FROM blocking.locktype
   AND blocked.database          IS NOT DISTINCT FROM blocking.database
   AND blocked.relation          IS NOT DISTINCT FROM blocking.relation
   AND blocked.transactionid     IS NOT DISTINCT FROM blocking.transactionid
   AND blocked.pid != blocking.pid
  JOIN pg_stat_activity blocking_act ON blocking.pid = blocking_act.pid
  WHERE NOT blocked.granted AND blocking.granted
)
SELECT * FROM lock_chain;
```

Build the chain. Group by `blocking_pid` → tree of waiters under each root blocker. Root blocker = blocking PID that is not itself blocked by anyone in the result set.

Edge cases.

- Long chains (A blocks B blocks C blocks D) — render up to 3 levels, collapse the rest with '+ N more waiters'.
- Cycle detection — Postgres deadlock detector kills one of them within `deadlock_timeout`, but the snapshot may briefly show a cycle. Mark visually.
- Permission to read `pg_stat_activity.query` — non-superusers see their own queries only. Show `<insufficient privilege>` for others.
- pg_locks can be huge on busy systems — push the join to the DB, never `SELECT * FROM pg_locks` into the client.

Render approach. Ink view with three levels: header (root blocker, warning icon), middle (the hold), waiters (collapsed list with duration). `k` opens a confirm modal: *"kill pid 18432 (jenkins@batch-worker, holding 14m 23s)? [y/N]"*.

Tests. Snapshot tests for the chain render. Integration tests against a Testcontainers Postgres that manufactures a real lock contention (`SELECT … FOR UPDATE` in one session, then `UPDATE` in another).

Performance. Refresh every 2s. If the lock count exceeds 500, switch to a "summary mode" that groups by table.

A2. `pgops top` — live slow-query view

Core query.

```
SELECT
  queryid,
  LEFT(query, 200)          AS query_excerpt,
  calls,
  total_exec_time           AS total_ms,
  mean_exec_time            AS mean_ms,
  stddev_exec_time          AS stddev_ms,
  rows,
  shared_blks_hit,
  shared_blks_read,
  100.0 * shared_blks_hit / NULLIF(shared_blks_hit + shared_blks_read, 0) AS cache_hit_pct
FROM pg_stat_statements
WHERE userid = (SELECT usesysid FROM pg_user WHERE usename = current_user)
ORDER BY total_exec_time DESC
LIMIT 100;
```

Edge cases.

- `pg_stat_statements` extension not installed → graceful error with install instructions.
- Queries truncated at `pg_stat_statements.track_utility_text_size` (default 1024 chars) — show … and link to full text via `queryid`.
- Reset events (`pg_stat_statements_reset()`) make the snapshot diff misleading — detect via the `stats_reset` column.
- High-cardinality query texts (think generated ORM queries with embedded literals) — show parameterized form, ignore literals when grouping by callsite.

Render approach. Sortable / filterable table. Press `s` to cycle sort (total → mean → calls → cache_hit). `/` to filter by table or query substring. Enter on a row opens an EXPLAIN view (see A7).

Tests. Property tests on the sort + filter logic. Integration against a seeded `pg_stat_statements`.

Performance. Poll every 2s. Cache the last 10 snapshots for delta math (calls/sec). For thousands of distinct queries, paginate.

A3. `pgops vacuum` — autovacuum doctor

Core query.

```
SELECT
  c.relname                                 AS table_name,
  s.schemaname,
  pg_size_pretty(pg_total_relation_size(c.oid)) AS size,
  n_live_tup,
  n_dead_tup,
  CASE WHEN n_live_tup > 0
       THEN ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup, 0), 2)
       ELSE 0 END                           AS dead_pct,
  last_vacuum,
  last_autovacuum,
  vacuum_count,
  autovacuum_count,
  age(c.relfrozenxid)                       AS xid_age,
  2147483647 - age(c.relfrozenxid)          AS xid_remaining
FROM pg_stat_user_tables s
JOIN pg_class c ON s.relid = c.oid
ORDER BY n_dead_tup DESC, age(c.relfrozenxid) DESC;
```

Edge cases.

- Wraparound-horizon math: tables nearing `autovacuum_freeze_max_age` need an emergency vacuum.
- TOAST tables — usually invisible to users but contribute to bloat. Include them in detail view.
- Partitioned tables — `pg_stat_user_tables` rows per partition. Group by parent for the summary view.

Render approach. Two-pane: table list on the left (sorted by bloat-risk), per-table detail on the right (history of vacuums, wraparound bar). Advise output: *"run VACUUM ANALYZE users — 42% dead tuples, last successful 14 days ago."*

Tests. Snapshot tests on the advise output. Integration against a Postgres with synthetic dead tuples.

Performance. This is a one-shot query, not polling. Refresh on `r`.

A4. `pgops slots` — logical replication slot monitor

Core query.

```
SELECT
  slot_name,
  plugin,
  slot_type,
  active,
  active_pid,
  restart_lsn,
  confirmed_flush_lsn,
  pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn) AS lag_bytes,
  pg_size_pretty(
    pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)
  ) AS lag_pretty
FROM pg_replication_slots
ORDER BY lag_bytes DESC NULLS LAST;
```

Disk-exhaustion forecast. Track `restart_lsn` over time per slot. Compute `bytes_per_minute` from the last 10 polling intervals. Project against free disk space (`SELECT pg_size_pretty(pg_database_size(current_database()))` and the OS-level free space if available). Output: *"Slot `etl_to_snowflake` at 12.4 GB lag, growing 80 MB/min. Disk-exhaustion ETA: 14h 22m."*

Edge cases.

- Slot is inactive but holding WAL — the killer scenario. Flag in red with a "consider dropping" suggestion.
- `confirmed_flush_lsn` may be null on physical slots — handle separately.
- Cross-version `pg_wal` vs `pg_xlog` (older PG) — detect and switch.

Render approach. Slot list + per-slot detail with lag history sparkline (Unicode block chars work in TUI). Snapshot exports include the ETA.

Tests. Forecast math unit tests against synthetic data series.

A5. `pgops parts` — partition manager

Core query (pg_partman introspection).

```
SELECT
  parent_table,
  partition_interval,
  premake,
  retention,
  retention_keep_table,
  last_partition,
  inherit_privileges,
  template_table
FROM partman.part_config;
```

Plus per-table partition health:

```sql
SELECT
  inhrelid::regclass AS partition_name,
  pg_get_expr(relpartbound, inhrelid) AS partition_bound,
  pg_size_pretty(pg_total_relation_size(inhrelid)) AS size,
  (SELECT MAX(reltuples)::bigint FROM pg_class WHERE oid = inhrelid) AS row_estimate
FROM pg_inherits
JOIN pg_class ON pg_inherits.inhrelid = pg_class.oid
WHERE inhparent = $1::regclass
ORDER BY partition_bound;
```

Edge cases.

- Tables not yet on pg_partman — provide a wizard to migrate (`partman.create_parent(...)` with sensible defaults).
- Gaps in the partition sequence (range partitioning missed a month) — detect by parsing `partition_bound`.
- Detached partitions still on disk — list separately.

Render approach. Parent-tables list. Drill into a parent → list of partitions with health (date range, size, row count). Actions: `m` runs `partman.run_maintenance()` with preview; `c` opens the migrate wizard.

A6. `pgops review` — pre-migration safety check

Parse layer. libpg_query compiled to WASM. Feed it the SQL file → AST.

Detection rules. Walk the AST. Patterns to flag:

Pattern	AST signature	Risk
`ADD COLUMN … NOT NULL` without `DEFAULT` (or with non-constant default)	`AlterTableStmt` → `AddColumn` → `Constraint NOT NULL`	Full-table rewrite under ACCESS EXCLUSIVE
`ALTER COLUMN … TYPE`	`AlterTableStmt` → `AlterColumnType`	Full rewrite
`CREATE INDEX` without `CONCURRENTLY`	`IndexStmt` with `concurrent: false`	Blocks writes for index build
`DROP COLUMN` (non-rename)	`AlterTableStmt` → `DropColumn`	Locks metadata, hides bugs
`ALTER TYPE … ADD VALUE` mid-transaction	`AlterEnumStmt` inside `BEGIN`	Not allowed in PG <= 11

Rewrite output. For each finding, emit the safe multi-step sequence as markdown. (LLM optional, but template-driven is the v0 — predictable, no API key required.)

Edge cases. SQL inside DO blocks; conditional `IF EXISTS`; CTEs; psql \ commands (ignore).

Tests. Golden-file tests per pattern with input SQL + expected rewrite output.

A7. `pgops explain` — local web for plans

Plan ingest. Accepts JSON plan from stdin, file, or `pgops top` handoff (`Enter` on a query → `pgops explain` opens with that plan). Parse with the standard Postgres EXPLAIN JSON format.

Render. React + Vite, served from a local HTTP server on `127.0.0.1:7878`. d3-flame-graph for the time-weighted flame view; collapsible tree for the plan structure. Side-by-side toggle.

Edge cases.

- Plans with subplans / CTEs — render as nested groups.
- `EXPLAIN (FORMAT TEXT)` input — convert to JSON via libpg_query? Or refuse and ask for JSON. v0: refuse with helpful message.
- Buffers vs no buffers — both views work, but the cache-hit panel only shows when `buffers: true`.

Tests. Static HTML output diff against a fixed set of plan JSON files.

A8. `pgops events` — cloud event correlator

AWS adapter. Uses the AWS SDK v3 with the user's existing AWS profile.

```
import { RDSClient, DescribeEventsCommand } from '@aws-sdk/client-rds'
import { EC2Client, DescribeInstanceStatusCommand } from '@aws-sdk/client-ec2'

const events = await rds.send(new DescribeEventsCommand({
  SourceIdentifier: dbInstanceId,
  SourceType: 'db-instance',
  StartTime: aroundTime - windowSecs,
  EndTime: aroundTime + windowSecs,
}))
```

Categorize events by severity (failover = critical, backup-window-start = info).

EKS adapter. Same shape via `@kubernetes/client-node` → `events` API.

Render. Timeline view — events in a vertical list, color-coded by severity, with the relative time to the `--around` anchor.

Edge cases.

- No AWS profile available → graceful "skipping cloud event correlation, run with `AWS_PROFILE=...`"
- Cross-region clusters → require `--region`.
- Rate limits — AWS DescribeEvents is throttled. Cache per-time-window.

E. Architecture decision log (ADR)

Format: each decision gets a short record with context, choice, and consequences.

ADR-001: Ink (React for terminals) over blessed

Context. Two viable TUI frameworks for Node — Ink and blessed. **Decision.** Ink. **Consequences.** React mental model (easier contributions, larger pool of contributors), reactive re-renders, better testing tooling (ink-testing-library). Larger bundle size than blessed; for an `npx`-installed CLI, this is irrelevant.

ADR-002: One binary, not separate npm packages

Context. Could ship `pglocks`, `pgtop`, `pgvacuum` etc as separate packages. **Decision.** One binary with subcommands. **Consequences.** Cross-promotion (every README mentions siblings), shared connection / Ink / snapshot code (~60% reuse), one install. Cost: larger binary, more surface for maintainership. Mitigated by tree-shakeable subcommand modules.

ADR-003: Polling, not `LISTEN/NOTIFY`

Context. Postgres supports `LISTEN/NOTIFY` for push, but `pg_stat_statements` / `pg_locks` aren't pushable. **Decision.** Poll every 2 seconds (configurable). **Consequences.** Predictable load on the DB. Simple to reason about. 2-second latency is fine for ops use. Cost: a bit of constant query traffic — but the queries are cheap.

ADR-004: Read-only RBAC defaults

Context. Write tools (kill, vacuum) need elevated privileges. Reads only need `pg_monitor` role. **Decision.** Default to read-only mode. Write tools require explicit grants the user already has. **Consequences.** Safe to install on production. Documented Postgres role with minimum grants. Operators can opt-in to write tools per-command.

ADR-005: Shared connection pool across subcommands

Context. Each subcommand could open its own connection. **Decision.** One connection per process via a shared pool (`pg.Pool` with `max: 2`). **Consequences.** Lower connection-count impact on the DB. Cleaner shutdown. Cost: subcommand modules can't assume connection isolation.

ADR-006: MCP server as a thin wrapper, not a separate process

Context. MCP support could be a separate package or process. **Decision.** `pgops mcp` is a subcommand that boots an MCP server using the same query layer as the TUIs. **Consequences.** No duplicate logic. Same data, two surfaces (human TUI + agent MCP). Cost: MCP server lifecycle is now part of the same binary's surface area.

ADR-007: libpg_query-wasm for SQL parsing

Context. Regex parsing is unreliable for SQL. Need the real PostgreSQL parser. **Decision.** libpg_query compiled to WebAssembly. **Consequences.** Real Postgres parse fidelity for CTEs, multi-statement files, dialect quirks. Cost: ~2 MB WASM blob; lazy-load when `pgops review` is invoked.

ADR-008: No telemetry by default

Context. Many OSS tools collect anonymous usage telemetry. **Decision.** Opt-in only. Off by default. Local-only when on (the user runs `pgops mcp --analytics` and `pgops` writes to a local file they can inspect). **Consequences.** Trust by default. We learn less about usage than we could — fine, given the ops-tool audience.

F. Testing strategy

Layered tests

1. **Unit** — query builders, AST walkers, forecast math. Vitest. Fast feedback loop.
2. **Snapshot** — TUI renders via `ink-testing-library`. Golden-file diffs.
3. **Integration** — real Postgres via Testcontainers for Node. One container per test suite; reset state between tests.
4. **End-to-end** — spawn the CLI binary, drive keyboard, assert on stdout. Slower; reserved for smoke tests of each subcommand.
5. **Load** — does `pgops locks` stay responsive on a DB with 5,000 idle connections? `pgops top` with 100k query digests in `pg_stat_statements`?

Postgres version matrix

CI matrix runs integration tests against PG 13 / 14 / 15 / 16 / 17 / 18. Drop PG 12 (EOL November 2024). Add new majors within a month of release.

Note: queries differ across versions — `pg_wal_lsn_diff` vs `pg_xlog_location_diff`, `total_exec_time` vs `total_time`. Version-detect at connect and route to the right query.

TUI snapshot tests

```
import { render } from 'ink-testing-library'
import { LockChain } from './ui/LockChain'

test('renders root blocker with 3 waiters', () => {
  const { lastFrame } = render(<LockChain chain={mockChain} />)
  expect(lastFrame()).toMatchSnapshot()
})
```

Snapshots are committed. Diff in CI fails on unintended render changes.

Integration scaffolding

```
import { PostgreSqlContainer } from '@testcontainers/postgresql'

const container = await new PostgreSqlContainer('postgres:17')
  .withCommand(['-c', 'shared_preload_libraries=pg_stat_statements'])
  .start()

const dbUrl = container.getConnectionUri()
// run pgops against this URL
```

Performance budgets

- `pgops locks` first render: < 200ms after connect.
- `pgops top` poll cycle: < 100ms for 1k query digests.
- Memory: < 50MB resident for the longest-running TUI (24-hour `pgops top` session).

CI runs benchmarks on every PR; fails on regression $> 20\%$.

G. Security posture

RBAC requirements per subcommand

Document the minimum Postgres role grants. Provide a `pgops grants` subcommand that prints the SQL to create a least-privilege role.

Subcommand	Minimum role	Reason
`locks` (read-only)	`pg_monitor` + `pg_read_all_stats`	Reads `pg_locks`, `pg_stat_activity`
`locks` (with kill)	Above + `pg_signal_backend`	`pg_terminate_backend()`
`top`	`pg_monitor` + `pg_stat_statements` access	Reads the extension's view
`vacuum` (read-only)	`pg_monitor`	Reads `pg_stat_user_tables`, `pg_class`
`slots`	`pg_monitor` + `pg_read_all_settings`	`pg_replication_slots` + WAL position
`parts` (read-only)	`USAGE ON SCHEMA partman` + table read	pg_partman tables
`parts` (write)	Above + `EXECUTE ON FUNCTION partman.*`	run_maintenance, create_parent
`review`	None — local SQL parser, no DB connection	—
`explain`	None for `--serve` mode; SELECT on relations for live EXPLAIN	—
`events`	AWS / k8s creds; no DB grants needed	—

Audit log format

Every write action (`kill`, `vacuum`, `run_maintenance`) writes a record:

```
{
  "timestamp": "2026-05-30T14:32:47Z",
  "user": "alice@bastion-host",
  "action": "kill",
  "subcommand": "locks",
  "target": { "pid": 18432, "user": "jenkins", "app": "batch-worker" },
  "reason": "blocking 4 waiters on users for 14m 23s",
  "result": "terminated"
}
```

Written to `~/.pgops/audit.log` by default; configurable to syslog or stdout for shipping to SIEM.

Threat model for the kill action

- **Accidental kill.** Confirm modal with PID + user + app name + duration. Require `y` (not Enter).
- **Wrong target.** Display the held lock + query before confirming. Make it obvious which PID will die.
- **Cascade kill.** Killing a blocker shouldn't auto-kill waiters. Always one-by-one.
- **Malicious operator.** The user already has the grants needed to kill via `pg_terminate_backend()`. We don't add new attack surface; we make existing power easier to use. Audit log is the mitigation.
- **Shell injection.** No `kubectl` or `psql` shell-out anywhere. All DB ops via typed `node-postgres`.

Local-only `--serve`

Every subcommand with a `--serve` flag (`pgops explain --serve`, future ones) binds to `127.0.0.1` only. Documented; tested with a CI assertion. No remote-access mode in v0.

Secrets handling

- `DATABASE_URL` read from env or `-d` flag. Never logged.
- Audit log redacts the password if it appears in error messages.
- Snapshot exports omit any field that looks like a credential (regex match on common patterns).

Telemetry

Off by default. If on, local-only logging to `~/.pgops/usage.log`. No network calls for telemetry without explicit `--telemetry-endpoint` configuration.

Part 2 — Non-technical

B. Build plan with weekly milestones

v0 — `pgops locks` standalone (weeks 1–4)

Ship as a focused single tool. Name candidates: `pglocks`, `pgwait`. Don't pre-commit to pgops.

Week 1 — Vertical slice. - Day 1–2: Repo scaffold, README placeholder, npm package, license (Apache-2.0). Postgres connection via node-postgres. Lock chain query (A1 above). - Day 3–5: Ink TUI with the root-blocker tree render. Basic key bindings (`q` to quit). - Day 6–7: Kill action with confirm modal + audit log entry. - **Pass / fail:** by Sunday evening, you can `npx pglocks` against a real Postgres and see the lock chain.

Week 2 — Polish for launch. - Day 8–9: Snapshot export (`s` key) to stdout markdown. - Day 10–11: README rewrite around the demo. Asciinema recording. 60-second Loom. - Day 12–14: Publish to npm. Homebrew formula. Tag v0.1.0. - **Pass / fail:** by Sunday, a stranger could install in under 5 minutes from your README alone.

Week 3 — Launch. - Day 15: HN Show post, Tuesday 9 AM PT. Title: *"Show HN: pglocks — a TUI for Postgres lock chains during incidents."* - Day 15: Cross-post to r/PostgreSQL with the asciinema gif. - Day 16: Pitch Nikolay (Postgres.AI) on Twitter for a Postgres Weekly mention. - Day 17–21: Respond to every issue / DM / comment within 6 hours. Ship a fix daily. - **Pass / fail:** 100+ stars and 3+ stranger issues by end of week.

Week 4 — Iterate. - Day 22–28: Address the top 5 user-reported issues. Add one user-requested feature only if 3+ people asked for it. - **Gate:** 500+ stars and 5+ stranger issues = success → proceed to v1. Less than that = polish what's shipped, don't expand.

v1 — Rebrand to pgops, add `top` + `vacuum` + `review` (weeks 5–12)

Eight weeks, three new subcommands at ~2.5 weeks each.

Weeks 5–7: `pgops top`. Most demand, easiest port (similar shape to `locks`). **Weeks 8–10:** `pgops vacuum`. New territory (per-table dashboard), needs the Prom exporter sibling. **Weeks 11–12:** `pgops review`. Different shape (no TUI, just CLI + markdown output). libpg_query-wasm integration is the work.

End-of-v1 gate: 2k+ stars total, 20+ external issues, 1+ blog post by a non-friend.

v2 — Add `parts` + `slots` + `explain` (weeks 13–18)

Six weeks, three subcommands at 2 weeks each. By now the toolkit has a story; the marginal subcommands extend the audience.

v3 — Add `events` + `mcp` (weeks 19–24)

Six weeks. `events` is the cloud correlator. `mcp` wraps everything for agents — completes the modern-ops-toolkit story.

Gates

- **End of week 4:** 500+ stars or stop expansion.
- **End of week 12:** 2k+ stars or stop adding subcommands; polish what's there.
- **End of week 24:** 5k+ stars or rethink whether to maintain solo or open up to co-maintainers.

C. Naming and branding

v0 standalone name

Candidates:

- `pglocks` — descriptive, instantly understandable, easy to search for. The verb is implicit.
- `pgwait` — focuses on the waiter rather than the lock. Slightly more emotional. Less search-discoverable.
- `pgblock` — focuses on the blocker. Confusing with "block" the storage term.

Decision: `pglocks`. Maximal discoverability. The README does the emotional work.

Rebrand trigger

Rename to `pgops` when:

- v0 hit 500+ stars.
- At least one user has asked "what else do you build?"
- You have v1's first new subcommand (`pgops top`) ready to ship.

Don't rebrand before you have the second subcommand. The umbrella only makes sense when there's more than one thing under it.

Brand voice

The README sounds like a Postgres veteran wrote it for other veterans. Specific. Honest about limitations. Anti-marketing. Examples:

- **Do:** "Run during the incident. Stay in psql for ad-hoc queries."
- **Don't:** "Revolutionary Postgres ops platform."

Visual identity

Minimal. The brand IS the terminal screenshot. Don't ship a logo for v0. If the project takes off, commission a simple monospace wordmark for v1.

Color palette for the asciinema demo and screenshots:

- Background: `#0d1117` (terminal black)
- Foreground: `#e6edf3` (off-white)
- Critical: `#f85149` (red)
- Warning: `#d29922` (amber)
- Info: `#58a6ff` (blue)
- Success: `#3fb950` (green)
- Dim: `#8b949e` (slate)

These are the same palette as GitHub's dark theme — familiar to developers and tested for accessibility.

README template

```
# pglocks
> Find the lock blocker during a Postgres incident. In your terminal.

[60-second Loom embedded]
[asciinema gif fallback]

## Install
npx pglocks   # or
brew install pglocks

## Use
pglocks                  # connect via $DATABASE_URL
pglocks -d "postgres://..."
```

```
## What it does
[3 bullets, with screenshot of each]

## What it doesn't do
[Honest list, links to alternatives]

## License
Apache-2.0
```

H. Community and governance

License

Apache-2.0. Permissive, business-friendly, well-understood. Lets companies use it without compliance friction. Trades the AGPL "force-share modifications" hammer for broader adoption — correct for ops tooling.

Not MIT (silent on patents). Not AGPL (hostile to commercial users you want to attract).

Contribution guide

Three contribution paths, each documented:

1. **Issues & feedback** — open issues with reproductions. Most valuable contribution for the first 6 months.
2. **Bug fixes** — PR with test. Auto-merge if CI green and review approves.
3. **New subcommand or feature** — RFC issue first ("intent to build"), discuss scope, then PR. Avoids wasted work on rejected ideas.

`CONTRIBUTING.md` covers the dev loop: clone → `npm install` → `npm test` → submit. Testcontainers spins up Postgres automatically.

Subcommand-author program

If the toolkit grows past your bandwidth, open the subcommand surface for community additions:

- A subcommand author proposes via RFC issue with the workflow it serves.
- Accepted subcommands live in `packages/sub-<name>` with a single maintainer (the author).
- They get commit access to their package only, plus README credit.
- Core team (you) gates `core/`, `ui/`, and the binary entry point.

This model is how Babel scaled past one maintainer. It's optional — only adopt if the toolkit grows past one person.

Maintainership

For v0–v1: solo. Make this explicit in the README — "Maintained by [you]."

If a real contributor emerges by v2 (consistent PRs, good judgment), invite as co-maintainer. Document the criteria for adding maintainers.

Code of conduct

Adopt the Contributor Covenant 2.1. Standard, defensible, widely understood. Don't write a custom one.

Sponsorship / funding

Enable GitHub Sponsors from day 1. No expectation, just a path for people who want to support. Don't gate features behind sponsorship.

If revenue ever crosses ~$2k/month, set up an Open Collective for transparent fund handling.

J. Documentation strategy

Three audiences, three doc layers

1. **The hurried operator** (90% of users). Needs the demo, install command, and one example per subcommand. Lives in the README.
2. **The careful adopter** (9% of users). Wants per-subcommand docs (man-page style), the security posture, the RBAC grants. Lives in `docs/`.
3. **The contributor** (1% of users, but they shape the future). Wants architecture, ADRs, test scaffolding. Lives in `CONTRIBUTING.md` + ADR records.

README anatomy

The README is the launch artifact. Optimize for the hurried operator:

```
# pgops
> One-line value prop, no marketing.

[Demo asciinema or Loom, embedded above the fold]

## Install (one line)
## Use (one example per subcommand, with screenshot)
## What it doesn't do (with links to alternatives)
## License
```

Everything else lives in `docs/`. Don't bloat the README with explanations; link to them.

Per-subcommand docs (man-page style)

Each subcommand gets `docs/<sub>.md` with:

- **Synopsis** — one line of usage.
- **Description** — what it does, in plain English.
- **Options** — flags + env vars.
- **Examples** — three real examples, increasing in complexity.
- **Reach for this when...** — the workflow trigger.
- **Doesn't do** — honest limitations + links to alternatives.

Recipes

Cross-subcommand workflows in `docs/recipes/`:

- *"Diagnose connection pool exhaustion in 3 commands"*
- *"Audit who has access to the users table"*
- *"Pre-flight a migration safely"*
- *"Postmortem from snapshots"*
- *"Run pgops during an incident from your phone over SSH"*

Each recipe is ~200 words + commands. These become the most-shared content on Reddit / HN.

Tutorials

- **First 5 minutes** — get the user from zero to one successful `pgops locks` against their local Postgres.
- **First useful action** — get them to use `pgops` to solve a real problem in their own environment within an hour.

Both as written docs + asciinema. Video is optional.

asciinema gallery

Each subcommand has a 30-second asciinema cast in `docs/casts/`. Embed in the per-subcommand doc. Easier to grok than written examples.

What NOT to document

- Internals of `node-postgres` (link to their docs).
- React / Ink primitives (link to Ink docs).
- Postgres concepts the audience already knows (don't explain pg_stat_statements; they're already running it).

Documentation discipline: every page answers "what does the operator need that they can't get elsewhere?"

Deferred sections (skipped from outline)

- **D. Distribution playbook** — channel-by-channel launch plan. Skipped for now; partially covered in B (weekly milestones, launch week).
- **I. Sustainability and exit** — graceful hand-off, commercial path. Skipped for now; revisit if the project hits sustained traction (post-v2).

Part 3 — Background (Postgres primer)

Domain-knowledge refresher embedded into the plan. For every concept pgops touches: what it is in plain English, what it reveals, and which subcommand uses it. Use it as a context reset when you've been away from Postgres for a while.

1. The Postgres mental model (the whole machine)

Before zooming in on the views pgops reads, here's Postgres as a whole — what kind of system it is, what its parts are, and where pgops sits in the picture.

1.1 What Postgres is

Postgres is several things at once:

- **A relational database engine.** SQL, tables, rows, indexes, joins, transactions.
- **A multi-process server.** One parent process (postmaster), one backend per client connection, plus background workers.
- **A transaction system.** ACID with MVCC (Multi-Version Concurrency Control) — readers never block writers and vice versa.
- **An extensible platform.** Extensions can add new data types, index types, foreign-data wrappers, even storage engines. `pg_stat_statements`, `pg_partman`, `pgvector`, PostGIS are all extensions.
- **A storage manager.** Heap tables, indexes, WAL, TOAST (oversized values), tablespaces.
- **A replication system.** Physical (byte-for-byte WAL streaming) and logical (decoded row changes), with slots tracking subscriber state.

1.2 Runtime — processes and memory

Postgres is process-based, not thread-based.

- **Postmaster.** The parent process. Accepts new connections, forks backends, supervises background workers.
- **Backend (one per connection).** Handles one client's queries. Lives for the duration of the connection.
- **Background workers** — autovacuum launcher and workers, checkpointer (flushes dirty buffers), WAL writer (flushes WAL), background writer (proactive page writes), stats collector, logical replication workers.
- **Shared memory** — `shared_buffers` (page cache shared across backends), WAL buffers, lock tables.
- **Per-backend memory** — `work_mem` (sorts, hashes), `maintenance_work_mem` (vacuum, CREATE INDEX).

Because connections are process-heavy, any serious workload runs a pooler in front (pgbouncer).

1.3 The data hierarchy

- **Cluster** — one Postgres installation. Hosts many databases.
- **Database** — isolated namespace. A backend talks to one at a time.
- **Schema** — a namespace within a database (default: `public`).
- **Table** — rows + columns, stored as a "heap" of 8 KB pages on disk.
- **Index** — separate data structure pointing into table rows. Btree (default), hash, GIN (full-text + arrays), GiST (spatial, ranges), BRIN (block-range, time-series), SP-GiST.
- **TOAST table** — auto-created sidecar storage for oversized values (> ~2 KB). Usually invisible to you.
- **Sequence** — a counter (powers SERIAL columns).
- **View / materialized view** — saved queries (live or cached).
- **Tablespace** — a physical location on disk; lets you put a table on a different volume.

All metadata about all of the above lives in the **system catalog** — itself a set of Postgres tables in the `pg_catalog` schema. Querying `pg_class` returns metadata about every table in the database, including `pg_class` itself.

1.4 The query lifecycle

When you send `SELECT ... FROM users WHERE ...`:

1. **Parser** — SQL text → syntax tree.
2. **Rewriter** — applies rules (views expand here; row-level security policies attach here).

3. **Planner / optimizer** — cost-based decision: which index, which join order, which join algorithm (nested loop / hash / merge), parallel or not.
4. **Executor** — walks the plan tree, producing rows.
5. **Result** — sent to the client.

`EXPLAIN` shows the plan; `EXPLAIN ANALYZE` runs it and shows actual times. Bad plans (`Seq Scan` on a huge table, nested loops over millions of rows) come from missing indexes, stale planner stats, or bad query shape.

1.5 Transactions and concurrency

- **ACID.** Atomicity (all or nothing), Consistency (constraints hold), Isolation (concurrent txns don't see each other's in-flight changes), Durability (committed = on disk via WAL).
- **MVCC.** Every row has hidden `xmin` / `xmax`. Readers see the version visible at their snapshot. Writers create new versions; old ones are "dead tuples" until vacuum reclaims them. The cost of "no read locks" is the housekeeping.
- **Isolation levels.** `READ COMMITTED` (default; each query gets a fresh snapshot), `REPEATABLE READ` (same snapshot for the whole txn), `SERIALIZABLE` (full serializability via predicate locking).
- **Lock modes.** 8 table-level modes, plus row-level and advisory locks. Detail in §3.1.
- **Deadlock detection.** After `deadlock_timeout` (default 1s) of waiting, Postgres checks for a cycle. If found, one txn is aborted.
- **Idle in transaction.** A session that ran `BEGIN`, did some work, then went idle. Its xid pins back autovacuum — single biggest cause of preventable bloat.

1.6 Replication and durability

- **WAL — Write-Ahead Log.** Every change is logged before it touches data files. Replay the WAL after a crash → durability. Stream the WAL → replication.
- **Checkpoints.** Periodically flush dirty shared buffers to data files. Bounds the crash-recovery window.
- **Physical replication.** Standby applies WAL byte-for-byte. Same Postgres version. Used for HA and read replicas.
- **Logical replication.** Postgres decodes WAL into row-level events (with column context). Streams to other Postgres, to Kafka via Debezium, to Snowflake via vendor tools.
- **Replication slots.** Per-subscriber bookkeeping; pins WAL until the subscriber confirms it's caught up. Orphan slots are the killer scenario (subscriber dead, slot still defined, WAL piles up, disk fills).
- **Backups.** `pg_basebackup` (full snapshot) + WAL archiving (continuous) = point-in-time recovery. Production tools: `pgbackrest`, `WAL-G`.

1.7 Routine maintenance

- **VACUUM.** Reclaims dead tuples (makes space reusable; doesn't necessarily shrink the file). Also freezes old xids (preventing wraparound). Non-blocking — usually.
- **ANALYZE.** Refreshes planner statistics in `pg_statistic` so the optimizer picks good plans.
- **Autovacuum.** Background workers running VACUUM + ANALYZE on tunable thresholds. Per-table overrides via `ALTER TABLE`.
- **REINDEX.** Rebuilds bloated indexes. `REINDEX CONCURRENTLY` (PG 12+) does it without blocking.
- **Wraparound horizon.** XIDs are 32-bit; after ~2 billion txns the counter wraps. If `relfrozenxid` ages past `autovacuum_freeze_max_age`, an emergency vacuum kicks in. If it ages past `2^31 - 1`, the DB shuts down to prevent data loss.

1.8 Observability

- **Stats collector → `pg_stat_*` views.** Per-table mutation counts, last vacuum, query counts. `pg_stat_activity` for live sessions.
- **`pg_stat_statements`.** Extension. Aggregates query timing across all calls. The standard tool for "which queries are slow."
- **`pg_locks`.** Live table of every lock currently held or waited on.
- **Logs.** Slow-query log (`log_min_duration_statement`), error log, csvlog / jsonlog for structured ingest.
- **`pg_stat_io`** (PG 16+), `pg_buffercache`, `pg_stat_progress_*` for live progress on vacuum / index build / etc.

1.9 Security

- **Roles.** No user-vs-group distinction in Postgres — both are roles. Roles can `INHERIT` from other roles.
- **GRANT / REVOKE.** Permissions on databases, schemas, tables, sequences, functions, columns.
- **Row-level security (RLS).** Policies that filter what rows a role can see / modify. Used heavily by Supabase and similar.
- **`pg_hba.conf`.** Host-based authentication — which IPs use which auth method (`md5`, `scram-sha-256`, `peer`, `cert`, `ldap`).
- **TLS / SSL.** `sslmode=require` on the client; `ssl_cert_file` / `ssl_key_file` on the server.

1.10 Extensions — how they work, how you use them

Part of what makes Postgres durable as a platform. Extensions are first-class — they can add data types, index types, functions, foreign-data wrappers, even storage engines. Many of Postgres's most-loved features are extensions (`pg_stat_statements`, PostGIS, TimescaleDB, pgvector).

The extension catalog (the ones to know)

- **`pg_stat_statements`** — query stats. Almost always enabled in production.
- **`pg_partman`** — partition management automation.
- **PostGIS** — geospatial types + indexes.
- **TimescaleDB** — time-series (hypertables, continuous aggregates).
- **pgvector** — vector similarity search for embeddings.
- **`pgaudit`** — audit logging.
- **PL/pgSQL** (always installed), **PL/Python**, **PL/Perl** — procedural languages for stored procedures.
- **`postgres_fdw`** — foreign data wrapper to other Postgres (query a remote Postgres as if it were a local table).
- **`pg_cron`** — schedule SQL via Postgres itself, no external cron.
- **`hypopg`** — hypothetical indexes (simulate an index without building it).
- **`pg_buffercache`** — what's in shared_buffers.
- **`uuid-ossp`**, **`citext`**, **`hstore`** — small utility extensions, ship with Postgres.

How extensions work mechanically An extension is a packaged set of: SQL files (creating tables / functions / types), an optional shared library (`.so`) loaded into Postgres at startup, and a control file (`<name>.control`) with metadata.

Three install layers (this is the part people get wrong):

1. **OS-level install** — extension binaries and SQL files land in Postgres's extension directory. Either via package (`apt install postgresql-15-pg-partman`), built from source, or pre-bundled with managed Postgres.
2. **Cluster-level activation** — extensions that hook into Postgres internals (`pg_stat_statements`, `pgaudit`, `pglogical`) need their library name added to `shared_preload_libraries` in `postgresql.conf`. **This requires a Postgres restart.** Skipping the restart is the #1 "I installed it and it doesn't work" cause.
3. **Database-level activation** — `CREATE EXTENSION <name>;` creates the extension's objects in the *current database*. **Repeat per database** — `pgvector` in `app_db` is NOT available in `app_db_test`.

User workflows that involve extensions **Workflow: enabling pg_stat_statements for the first time.**

1. Edit `postgresql.conf`: `shared_preload_libraries = 'pg_stat_statements'`.
2. Restart Postgres.
3. `CREATE EXTENSION pg_stat_statements;` in each database you want to track.
4. Optionally tune: `pg_stat_statements.max = 10000`, `pg_stat_statements.track = 'all'`.
5. Wait for data to accumulate, or `SELECT pg_stat_statements_reset();` to start fresh.
6. `pgops top` now has data.

Workflow: setting up partition management.

1. Install `pg_partman` (cloud-managed Postgres usually has it; self-hosted: `apt install`).
2. `CREATE SCHEMA partman; CREATE EXTENSION pg_partman WITH SCHEMA partman;`
3. Create the parent table: `CREATE TABLE events (id bigserial, created_at timestamptz NOT NULL) PARTITION BY RANGE (created_at);`

4. Hand to pg_partman: `SELECT partman.create_parent('public.events', 'created_at', 'native', 'daily');`
5. Schedule maintenance — either system cron or `pg_cron` extension: `SELECT cron.schedule('partman-maint', '0 * * * *', 'SELECT partman.run_maintenance();');`
6. `pgops parts` shows partition health and can run maintenance interactively.

Workflow: choosing a cloud Postgres.

Extension availability is a hard constraint when picking a managed provider:

- **AWS RDS / Aurora** — extensions whitelisted per major version. Check the supported list. `pg_stat_statements` is on by default; `pg_partman` supported on most majors; pgvector supported since 2024.
- **Supabase** — very permissive. pgvector, `pg_cron`, `pgaudit`, `pg_jsonschema`, many ML extensions enabled.
- **Neon** — most open-source extensions, branching-aware.
- **Crunchy Bridge** — nearly the full open-source ecosystem.
- **Self-hosted** — anything that compiles.

The mistake: commit to a managed Postgres before checking it supports the extension you'll need 6 months from now (e.g., pgvector for an AI feature, pgaudit for SOC2).

Workflow: Postgres major-version upgrade.

1. Read extension changelogs — does each one support the new major version?
2. Build the new cluster with all required extensions installed.
3. Run `pg_upgrade` (or logical replication for zero-downtime).
4. `ALTER EXTENSION <name> UPDATE;` for each extension to apply post-upgrade schema changes.
5. Restart, verify.
6. (For pgops users: `pgops vacuum` will show wraparound horizons reset, since the upgrade does a full freeze.)

Workflow: debugging "pgops top says no data."

1. `SELECT * FROM pg_extension WHERE extname = 'pg_stat_statements';` — installed?
2. If not: check `shared_preload_libraries`, restart, then `CREATE EXTENSION`.
3. If yes but empty: was it reset? Is `pg_stat_statements.track = 'all'`? Check directly with `SELECT * FROM pg_stat_statements LIMIT 1;`
4. pgops surfaces this exact diagnostic on startup if it can't find data.

Workflow: adding richer observability over time.

1. Decide what you need — `pg_stat_kcache` (CPU + IO per query), `hypopg` (simulate indexes), `pg_qualstats` (predicate stats), `pgstattuple` (real bloat measurement).
2. Install + activate (steps above).
3. Restart Postgres if `shared_preload_libraries` change.
4. pgops auto-detects supported extensions on startup and the advise output sharpens. Example: `pgops vacuum` uses `pgstattuple` for real bloat numbers when available, heuristic estimate otherwise.

Extensions pgops cares about

Extension	Required by	Why	Falls back to
`pg_stat_statements`	`pgops top`	Aggregated query stats	Hard requirement — refuses to run without it
`pg_partman`	`pgops parts`	Partition management API	Detects absence, prints install command
`pgstattuple`	`pgops vacuum` (advanced bloat)	Real bloat measurement vs estimate	Heuristic estimate from `pg_stat_user_tables`
`hypopg`	`pgops top` (future: index suggest)	Simulate indexes without building	Suggestion without validation
`pg_buffercache`	`pgops top` (future: cache-hit panel)	What's in shared_buffers	Skips the panel
`pg_stat_io` (PG 16+)	`pgops vacuum`, `pgops top` (future)	Per-operation IO stats	Skips IO panel

pgops never installs extensions itself. It surfaces what's missing, prints the install command, and tells you which subcommand needs it. The user (or their DBA) decides what to enable.

Common extension gotchas

- **`shared_preload_libraries` requires a restart.** Forgetting this is the #1 install failure.
- **`CREATE EXTENSION` is per-database.** Repeat in every database that needs it.
- **`search_path` matters.** If `pgvector` is in the `extensions` schema and `extensions` isn't in your `search_path`, the operators are invisible.
- **Version skew after `pg_dump` / restore.** Restoring on a newer cluster may need `ALTER EXTENSION ... UPDATE;` to apply new objects.
- **Cloud providers can drop extensions.** AWS RDS has removed extensions across major upgrades; read the release notes before upgrading.
- **Some extensions cost real resources.** `pg_stat_statements.max = 10000` with long query texts = several MB shared memory; `pgaudit` can balloon log volume; `pg_buffercache` scanning shared_buffers is briefly expensive on big servers.
- **Cluster-wide vs in-database.** Extensions like `pg_cron` are installed in *one* database (usually `postgres`) but schedule jobs across the cluster. Easy to misconfigure.

1.11 Where pgops sits in this map

pgops focuses on the runtime / locks / observability / replication-health / maintenance layers. It deliberately doesn't try to be every-Postgres-tool-in-one.

Postgres area	pgops touches?	Why or why not
Process model + memory tuning	No	OS / sizing concern, not on-call workflow
Data hierarchy (schemas / tables / indexes)	Read-only catalog introspection	We read metadata, don't modify
Query lifecycle	Yes — EXPLAIN, slow-query stats	`pgops top`, `pgops explain`
Transactions + concurrency + locks	Yes — incident-time	`pgops locks`
Replication + durability	Yes — slot health, lag forecast	`pgops slots`
Routine maintenance	Yes — vacuum debugging, partition rotation	`pgops vacuum`, `pgops parts`
Observability views	Yes — they're the primary inputs	All subcommands
Security / auth config	No	We document required grants, don't manage them
Extension installation	No	We require some (pg_stat_statements, pg_partman); don't install them
Backups	No	Use `pgbackrest` / `WAL-G`
Migration safety (pre-flight)	Yes — SQL parsing	`pgops review`
Cloud-side events (RDS / EKS)	Yes — correlation	`pgops events`

Read this as: **pgops is the on-call surface for Postgres ops.** Other concerns either have good tooling already (backups, security config, memory tuning) or sit in adjacent disciplines (perf modeling, infra, app-level monitoring) that warrant their own tools.

1.12 The mental model in one sentence

Postgres is a process-per-connection server that uses MVCC for concurrency, WAL for durability and replication, autovacuum to reclaim the dead tuples MVCC leaves behind, and a system catalog you can query like any other table — and the union of those four properties is what makes its operational surface interesting enough to need a toolkit like pgops in the first place.

2. The catalogs and stat views pgops reads

View / table	What it is	What it reveals	pgops subcommand
`pg_locks`	Live table — one row per lock currently held or waited on	Who holds what, who's waiting on what, lock mode, relation	`locks`
`pg_stat_activity`	One row per session	Current query text, state (active / idle / idle-in-txn), wait event, transaction age, application name, client IP	`locks`, `top`
`pg_stat_statements`	Aggregated query stats (extension; must be in `shared_preload_libraries`)	Per-query: calls, total time, mean time, rows, cache hit %, last-reset timestamp	`top`
`pg_stat_user_tables`	Per-table mutation + vacuum stats	Live tuples, dead tuples, last vacuum / autovacuum, vacuum count	`vacuum`
`pg_class`	Master catalog for tables, indexes, sequences	Size, `relfrozenxid` (wraparound age), storage type, partition info	`vacuum`, `parts`
`pg_replication_slots`	Logical + physical replication slot state	Slot name, plugin, active subscriber pid, `restart_lsn`, `confirmed_flush_lsn`, lag bytes	`slots`
`pg_partman.part_config`	pg_partman extension's partition manifest	Parent tables, partition interval, retention, premake (future partitions to keep prepared)	`parts`

Honorable mentions — not deeply used by pgops in v0, but you should know they exist:

- `pg_stat_io` (PG 16+) — per-IO-type stats: reads, hits, evictions, writebacks. The modern way to see "where is my I/O going."
- `pg_buffercache` (extension) — what's currently cached in shared buffers.
- `pg_indexes` / `pg_index` — index metadata.
- `pg_settings` — every Postgres parameter and its current value.
- `pg_database` — database-level metadata.
- `pg_stat_replication` — physical replication subscribers (the streaming standbys).
- `pg_stat_progress_vacuum` — what an in-flight VACUUM is doing right now.

3. Protocols and conventions

3.1 Lock modes

Postgres has 8 table-level lock modes. They form a conflict matrix — some pairs are compatible, others block. Weakest to strongest:

Mode	Acquired by	Conflicts with
ACCESS SHARE	`SELECT`	ACCESS EXCLUSIVE only
ROW SHARE	`SELECT FOR UPDATE / SHARE`	EXCLUSIVE, ACCESS EXCLUSIVE
ROW EXCLUSIVE	`INSERT`, `UPDATE`, `DELETE`, `MERGE`	SHARE, SHARE ROW EXCLUSIVE, EXCLUSIVE, ACCESS EXCLUSIVE
SHARE UPDATE EXCLUSIVE	`VACUUM` (non-FULL), `ANALYZE`, `CREATE INDEX CONCURRENTLY`, some `ALTER TABLE` variants	Itself + stronger
SHARE	`CREATE INDEX` (without CONCURRENTLY)	All row-writers and stronger
SHARE ROW EXCLUSIVE	Foreign key checks, some `ALTER TABLE`	Itself + most others
EXCLUSIVE	Rare (some refresh-materialized-view paths)	Everything except ACCESS SHARE
ACCESS EXCLUSIVE	`ALTER TABLE` (most variants), `DROP TABLE`, `TRUNCATE`, `REINDEX`, `CLUSTER`	**Everything — even SELECT**

ACCESS EXCLUSIVE is the dangerous one. A migration that takes it blocks every read and write on that table for the migration's duration. On a multi-million-row table, that's measurable downtime. `pgops locks` exists to show you when one is held, by whom, and what's waiting.

Postgres also has **row-level locks** (acquired by `SELECT FOR UPDATE` etc), tracked on the rows themselves, not in `pg_locks`. And **advisory locks**, which are application-defined and don't conflict with anything Postgres does — useful for app-level coordination.

3.2 WAL and replication

Physical replication. A standby server applies WAL byte-for-byte from the primary. Same Postgres major version required. Used for HA failover and read replicas.

Logical replication. Postgres decodes WAL into row-level change events (with table/column context) and streams them to subscribers. Subscribers can be other Postgres instances, Kafka via Debezium, Snowflake via various tools. Used for CDC (Change Data Capture).

Replication slot. Postgres-side bookkeeping for one subscriber. The slot tracks how far the subscriber has confirmed receipt; Postgres won't prune WAL past that point. **The killer scenario:** the subscriber dies but the slot remains defined. WAL piles up indefinitely, the disk fills, the primary crashes. `pgops slots` watches for this.

LSN — Log Sequence Number. A 64-bit position in the WAL stream. Printed in hex like `0/16B5A88`. Used to measure replication lag: `pg_wal_lsn_diff(pg_current_wal_lsn(), slot.confirmed_flush_lsn)` returns bytes behind.

3.3 EXPLAIN

`EXPLAIN <query>` prints the planner's execution plan without running the query. `EXPLAIN ANALYZE <query>` runs it and reports actual timings (use with care on slow queries).

A plan is a **tree of nodes**: `Seq Scan` (full table scan), `Index Scan`, `Index Only Scan`, `Bitmap Heap Scan`, `Hash Join`, `Nested Loop`, `Merge Join`, `Sort`, `Aggregate`, `Limit`, etc. Each node reports a cost estimate (planner's prediction in arbitrary units) and, with ANALYZE, actual rows + actual time.

Read the plan **bottom-up** — the most-indented nodes happen first.

Output format options: `(FORMAT TEXT)` (default), `(FORMAT JSON)`, `(FORMAT YAML)`, `(FORMAT XML)`. Add `(BUFFERS)` to see cache hits / disk reads per node — essential for diagnosing whether a slow query is CPU-bound or IO-bound. `pgops explain` consumes the JSON format.

3.4 Roles and grants pgops needs

Role	What it grants
`pg_monitor`	Umbrella read-only role; includes `pg_read_all_stats`, `pg_read_all_settings`, `pg_stat_scan_tables`
`pg_read_all_stats`	Read every `pg_stat_*` view including those normally restricted
`pg_read_all_settings`	Read every parameter, including ones with restricted visibility
`pg_signal_backend`	Send signals (including `pg_terminate_backend()`) to sessions owned by non-superusers
`pg_partman` schema	Read `partman.*` tables; execute `partman.run_maintenance()` (granted explicitly per the pg_partman docs)

pgops's read-only subcommands need `pg_monitor`. The `kill` action in `pgops locks` additionally needs `pg_signal_backend`. The `pgops grants` subcommand generates a least-privilege role you can `CREATE ROLE` and grant.

4. The DBA workflows pgops accelerates

Incident lock-chain discovery

Page fires: writes are timing out.

The ritual: `SELECT ... FROM pg_locks JOIN pg_stat_activity ...` to find every blocked PID and who's blocking it. Build the chain in your head. Inspect the blocker's query text — is it a long-running batch job, a stuck `idle in transaction` session, a runaway analytical query? Decide: wait it out, kill the blocker (`pg_terminate_backend(pid)`), or escalate.

`pgops locks` packages this into a TUI: root-blocker view, waiter cascade, query text, `k` to kill, `[s]` to snapshot for the postmortem.

Slow-query triage

APM says endpoint X is slow.

The ritual: query `pg_stat_statements` for the worst offenders by `total_exec_time` (or `mean_exec_time` if the query is rare but expensive). Run `EXPLAIN ANALYZE` on the suspects. Look for `Seq Scan` on large tables, nested loops over large inputs, missing indexes, bad join orders, wrong row estimates (planner stale). Identify the fix — usually an index, sometimes a query rewrite.

`pgops top` is this loop in a live TUI; press Enter on a row to drop into the EXPLAIN view.

Autovacuum debugging

Tables feel bloated, or autovacuum runs constantly without keeping up.

The ritual: query `pg_stat_user_tables` for high dead-tuple ratios. Check `pg_class.relfrozenxid` against `autovacuum_freeze_max_age` for wraparound horizon (the further from `2^31`, the worse). Inspect autovacuum config — `autovacuum_vacuum_scale_factor`, `autovacuum_naptime`, per-table overrides. Maybe run `VACUUM (VERBOSE)` to see what's stuck. Maybe tune `autovacuum_vacuum_cost_limit` if vacuum is being throttled too hard. Maybe kill an idle-in-txn that's holding back cleanup.

`pgops vacuum` is the triage TUI — bloat-ranked, freeze-age-aware, with advise output.

Migration safety — expand → backfill → contract

The pattern. Schema changes are done in **three steps**, not one.

1. **Expand** — add the new column (nullable) or new index (`CREATE INDEX CONCURRENTLY`). Takes a weak lock, doesn't block traffic.
2. **Backfill** — populate the new column in batches: `UPDATE ... WHERE id BETWEEN x AND y`, sleeping between batches. Small batches mean short row-level locks and short transactions — no bloat, no waiters.
3. **Contract** — add the `NOT NULL` constraint (after backfill is done) / drop the old column / swap the application to read from the new column.

The naive single-statement version (`ALTER TABLE users ADD COLUMN email TEXT NOT NULL DEFAULT '';`) takes ACCESS EXCLUSIVE for the entire default fill — on a 12M-row table, that's ~45 seconds of total outage on `users`.

`pgops review` is the pre-flight check that reads your migration file and warns when you've written the naive version.

Replication monitoring

Routine check, or: replication lag alert fires.

The ritual: `SELECT * FROM pg_replication_slots` — every slot, its `active` flag, its `confirmed_flush_lsn`, the lag bytes vs `pg_current_wal_lsn()`. Cross-check `pg_stat_replication` for the actual subscriber state. The killer pattern: `active = false` + non-trivial lag = an orphan slot pinning WAL. Drop it (`SELECT pg_drop_replication_slot('slot_name')`) only after you're sure the subscriber is truly gone.

`pgops slots` watches this with a time-to-disk-exhaustion forecast based on slot growth rate — the line that wakes you at 2am with enough lead time to act.

Partition rotation

Time-series tables (events, logs, metrics) partitioned by month / week / day.

The ritual: ensure future partitions exist (pg_partman's "premake" — keep N future partitions ready), drop old partitions per the retention policy, monitor for gaps (a missed month means inserts fail). Most teams run `partman.run_maintenance()` from cron + cross their fingers.

`pgops parts` introspects `partman.part_config`, shows partition health (size, row count, range), and runs `partman.run_maintenance()` with a preview of what will change before it commits.

5. Cross-reference — Postgres concept → pgops subcommand

If you forgot…	Refresh by reading	And the pgops subcommand is
What `pg_locks` is	§2 + §4 incident lock-chain	`pgops locks`
Why `pg_stat_statements` matters	§2 + §4 slow-query triage	`pgops top`
Dead tuples + MVCC + autovacuum	§1 + §4 autovacuum debugging	`pgops vacuum`
Replication slots and LSN	§2 + §3.2 + §4 replication monitoring	`pgops slots`
pg_partman + range partitioning	§2 + §4 partition rotation	`pgops parts`
Expand → backfill → contract	§3.1 lock modes + §4 migration safety	`pgops review`
EXPLAIN plan structure	§3.3 EXPLAIN	`pgops explain`
Postgres role grants for monitoring	§3.4 roles table	All read-only subcommands (`pg_monitor`); `locks` (kill) adds `pg_signal_backend`

Where to read more (Postgres docs canonical)

Topic	Chapter
MVCC, locking, deadlocks	Ch 13 — Concurrency Control
EXPLAIN, planner	Ch 14 — Performance Tips
Autovacuum, wraparound	Ch 25 — Routine Database Maintenance Tasks
Logical replication slots	Ch 30 — Logical Replication
WAL internals	Ch 30.6 — Replication Slots
Catalog reference	Ch 53 — System Catalogs
Stat views	§28.2 — The Statistics Collector
Roles + grants	Ch 22 — Database Roles